

Cache Memory

ANISHA M. LAL

Cache memory

- **Locality of reference(principle of locality)**

It is the phenomenon of the same value or related storage locations being frequently accessed.

Types of reference locality:

Temporal locality

- refers to the reuse of specific data and/or resources within relatively small time durations.

Spatial locality

- refers to the use of data elements within relatively close storage locations.

Sequential locality occurs when data elements are arranged and accessed linearly, e.g., traversing the elements in a one-dimensional array.

Cache memory

- Definition of Cache memory

The cache is a smaller, faster memory which stores copies of the data from the most frequently used main memory locations.

- made up of SRAM.
- Effectiveness of cache system depends on hit ratio.

Terminologies:

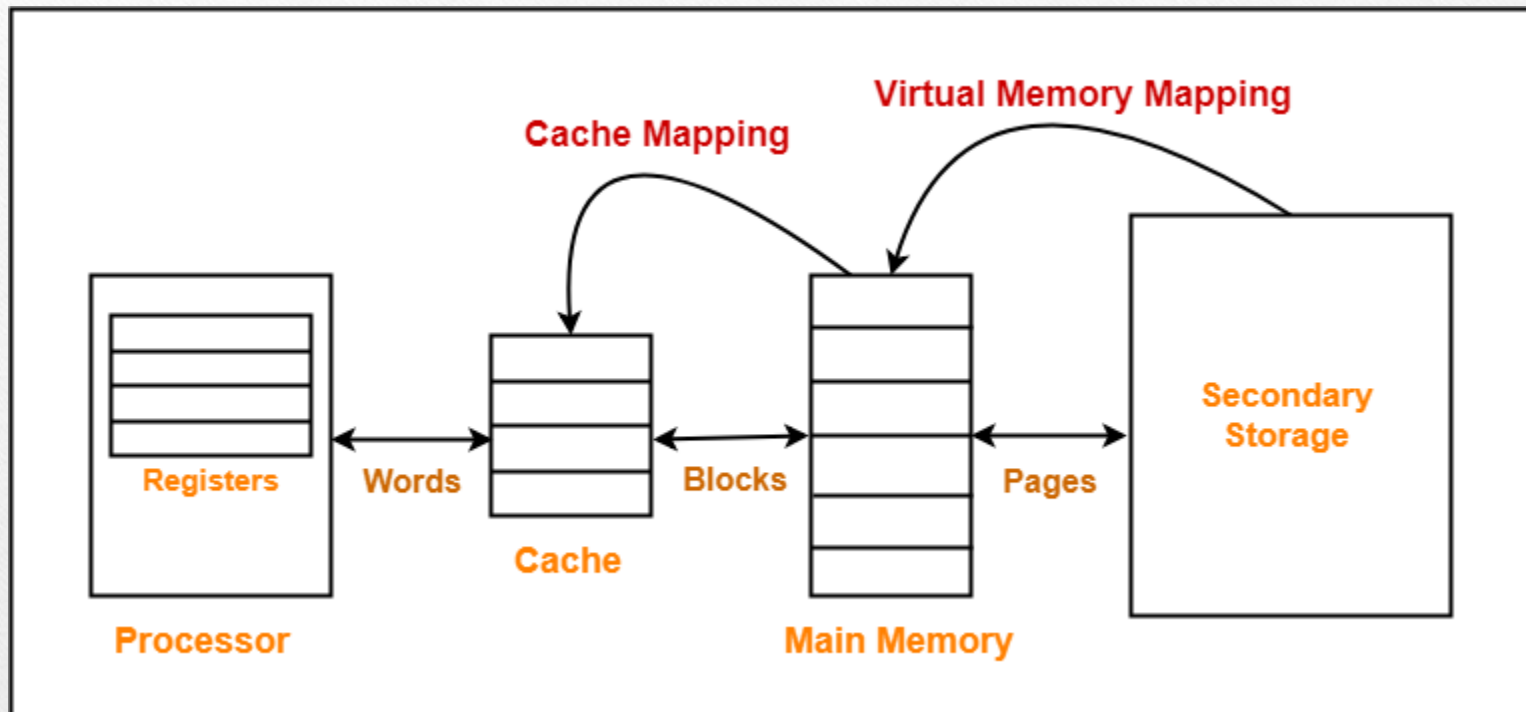
1. Cache hit – item found in cache
2. Cache miss – item not in cache

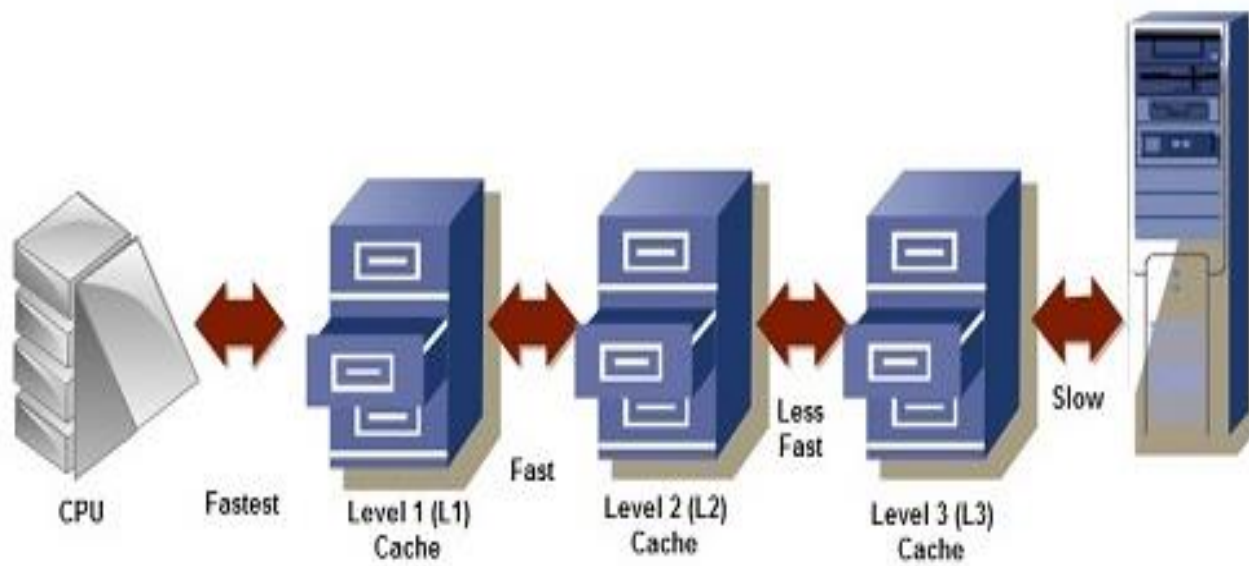
3. Hit ratio: ratio between number of hits to total number of memory references
4. Miss penalty: additional number of cycles required to serve the miss
5. Time required for the cache miss depends on both the latency and bandwidth
 - Latency – time to retrieve the first word of the block
 - Bandwidth – time to retrieve the rest of this block

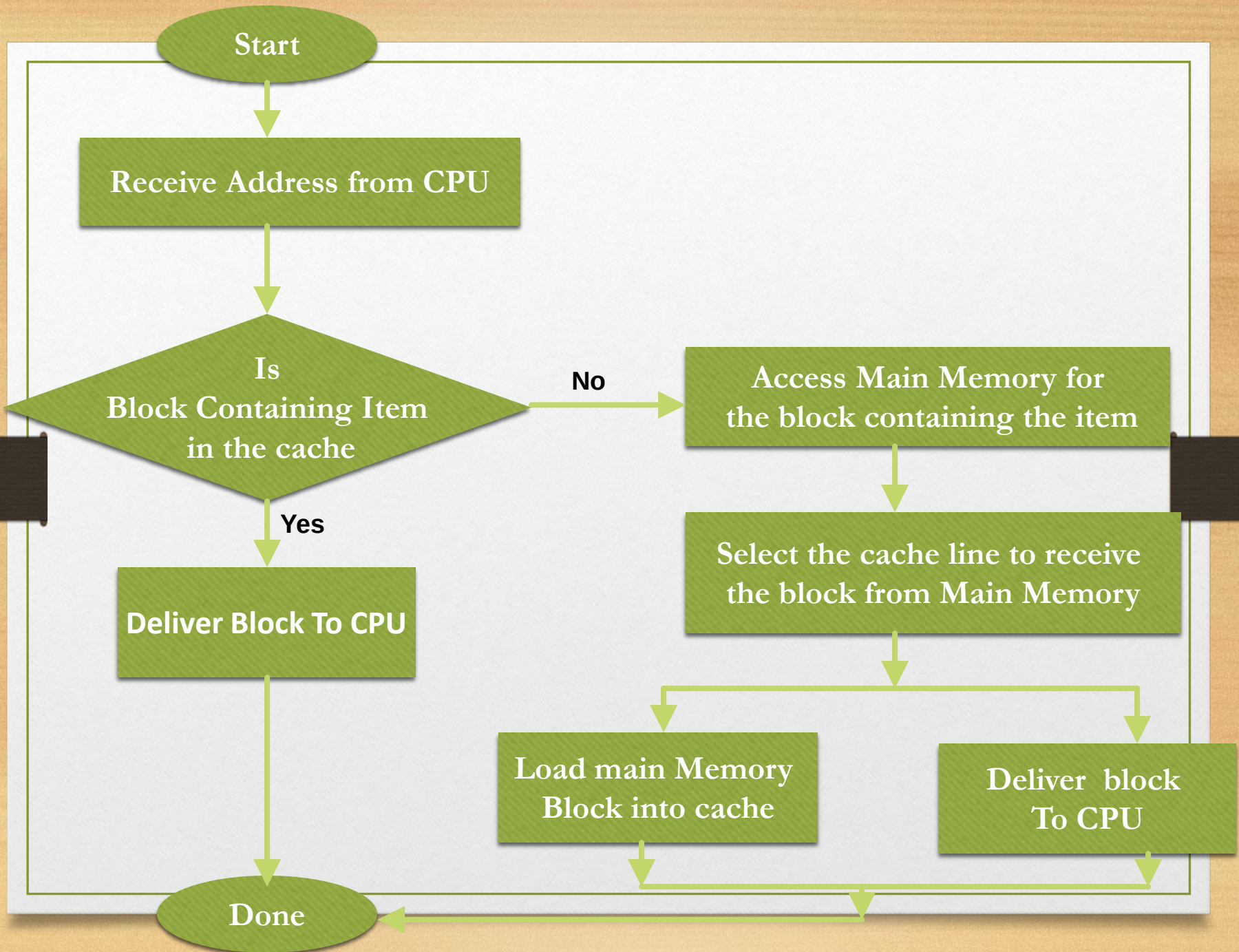
Blocks: group of words in main memory.

Block frame: term used to refer blocks in cache memory.[group of words in cache memory]

Number of words in blocks and block frames are same.







Cache Memory Management Techniques

- Block Placement
 - Direct Mapping
 - Set Associative
 - Fully Associative
- Block Identification
- Block Replacement
- Update Policies

Cache Memory Management Techniques

```
graph LR; A[Cache Memory Management Techniques] --> B[Block Placement]; A --> C[Block Identification]; A --> D[Block Replacement]; A --> E[Update Policies]; B --> B1[Direct Mapping]; B --> B2[Set Associative]; B --> B3[Fully Associative]; C --> C1[Tag]; C --> C2[Index]; C --> C3[Offset]; D --> D1[FCFS]; D --> D2[LRU]; D --> D3[Random]; E --> E1[Write Through]; E --> E2[Write back]; E --> E3[Write around]; E --> E4[Write allocate];
```

Block Placement

Direct Mapping

Set Associative

Fully Associative

Block Identification

Tag

Index

Offset

Block Replacement

FCFS

LRU

Random

Update Policies

Write Through

Write back

Write around

Write allocate

Direct Mapping

- Each block has only one place it can appear in the cache.
- No replacement policies are used.
- simpler, but much larger than an associative one to give comparable performance

Cache block = (Block address) mod (number of block in cache)

Direct Mapping

15	
14	
14	
13	
12	
11	
10	
9	
8	
7	
6	
5	
4	
3	
2	
1	
0	

	7
	6
	5
	4
	3
	2
	1
	0

Cache

(MM Block address) mod (Number of lines in a cache)

$$(12) \bmod (8) = 4$$

Main Memory

Direct mapping -example

- Consider the following configuration of system:
Main memory – 32 blocks
cache memory – 8 block frames
cache is empty and the block referred by CPU is 12.
Find the block frame of cache where the requested block is placed.

- Cache block frame = (Block address)mod(number of block in cache)

$$= 12 \bmod 8$$

$$= 4$$

Now if the block referred by CPU is 20.

Advantage- simplicity in determining where to place an incoming main memory block in the cache.

Disadvantage : inefficient use of the cache.

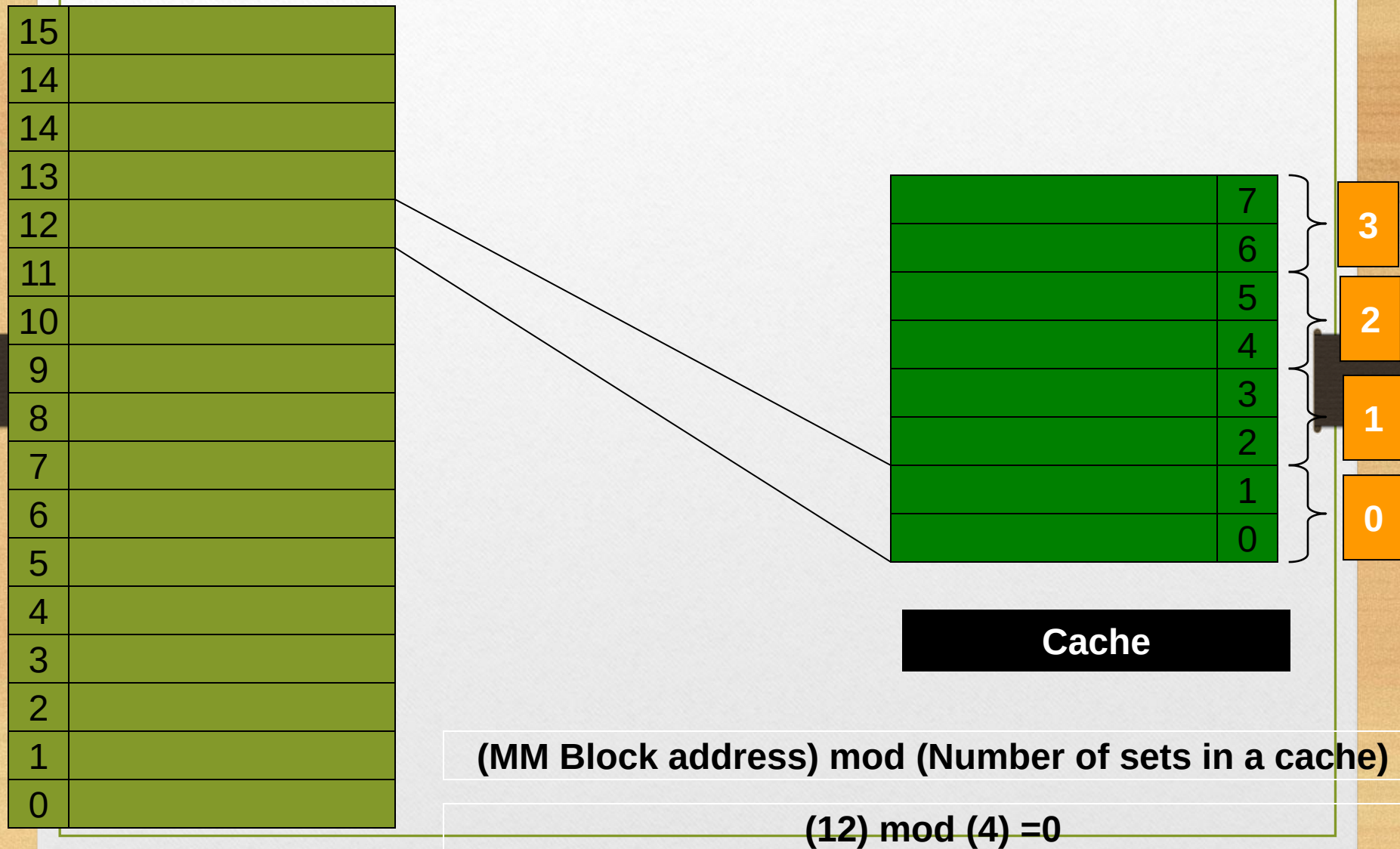
Leads to low cache hit ratio.

Consider, for example, the case of a main memory consisting of 4K blocks, a cache memory consisting of 128 blocks, and a block size of 16 words.. For example, main memory blocks 0, 128, 256, 384, . . . , 3968 map to cache block 0. We therefore call the direct-mapping technique a **many-to-one mapping technique**.

Set Associative cache

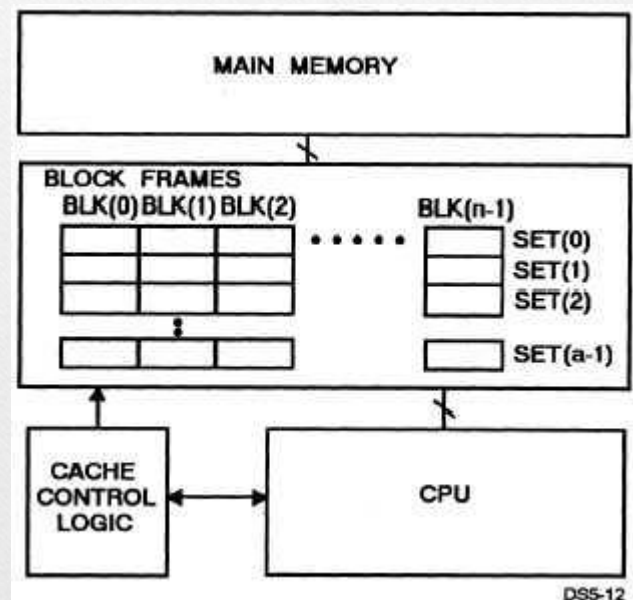
- Blocks are placed in a restricted set of places in the cache.
- Set : group of blocks.
- Set is chosen: $(\text{block address}) \bmod (\text{number of sets in the cache})$
- N way set Associative: if there are n blocks in the set

Set Associative Mapping



Main Memory

N way set associative



Example

- Consider the following configuration of system:

Main memory – 32 blocks

cache memory – 8 block frames

Number of sets = 4 each with 2 blocks

cache is empty and the block referred by CPU is 12.

Find the block frame of cache where the requested block is placed.

Example

- Set number= (Block address)mod(number of sets in cache)

$$= 12 \bmod 4$$
$$= 0$$

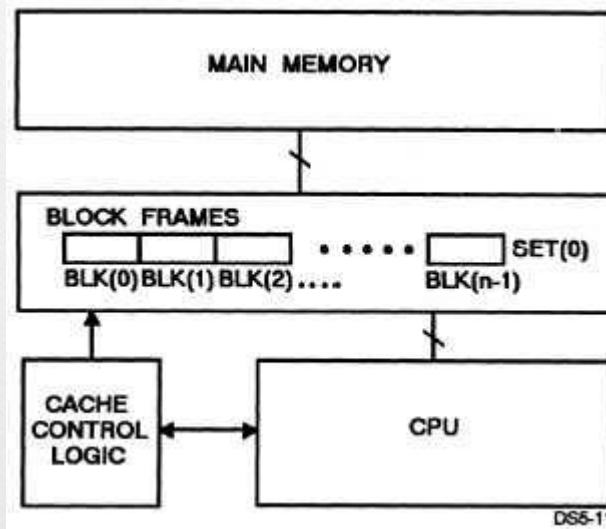
12th block can be placed in either in 0th or 1st block frame of set 0.

- First map is to map the set in the cache and then to the corresponding block frame.

Fully Associative

- incoming main memory block can be placed in any available cache block.
- The block 12 (for previous example) can be placed in any one of the available block frames.
- Replacement algorithm is used.

Fully Associative



Fully Associative Mapping

15	
14	
14	
13	
12	
11	
10	
9	
8	
7	
6	
5	
4	
3	
2	
1	
0	

	7
	6
	5
	4
	3
	2
	1
	0

Cache

Random

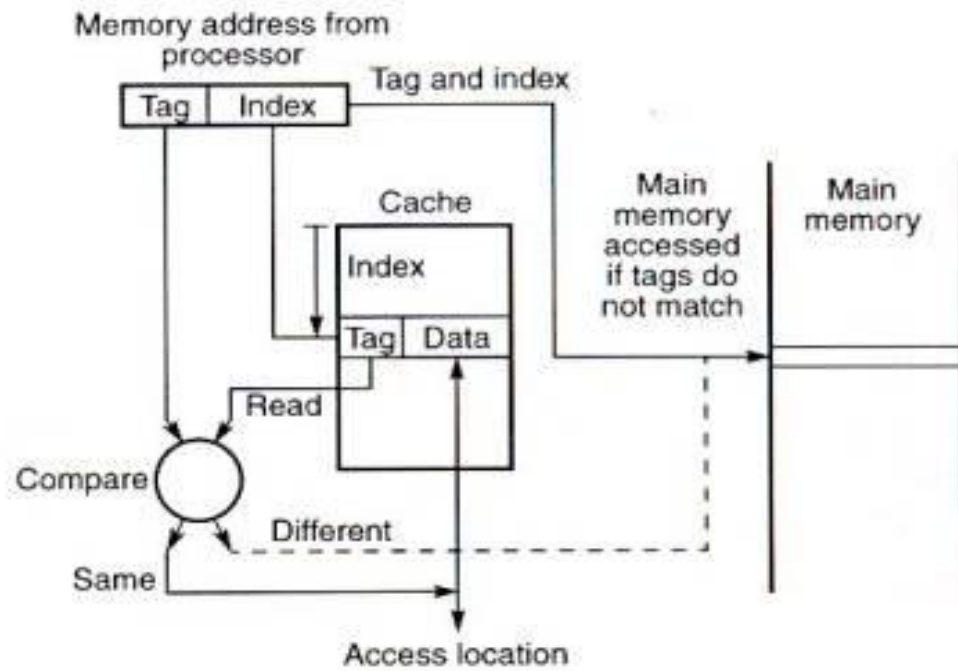
Main Memory

Associative Mapping

- Fastest, most flexible but very expensive
- Any block location in cache can store any block in memory
- Stores both the address and the content of the memory word
- CPU address of 15 bits is placed in the argument register and the associative memory is searched for a matching address
- If found data is read and sent to the CPU else main memory is accessed.
- CAM – content addressable memory

Identification of block for Direct mapped caches

- The sizes of memories are typically expressed in power of 2.
- Main memory size $= 2^m$
- Block size $= 2^n$
- The number of block in the main memories is $= 2^{m-n}$
- The cache size $= 2^k$
- Number of lines in the cache $= 2^{k-n}$

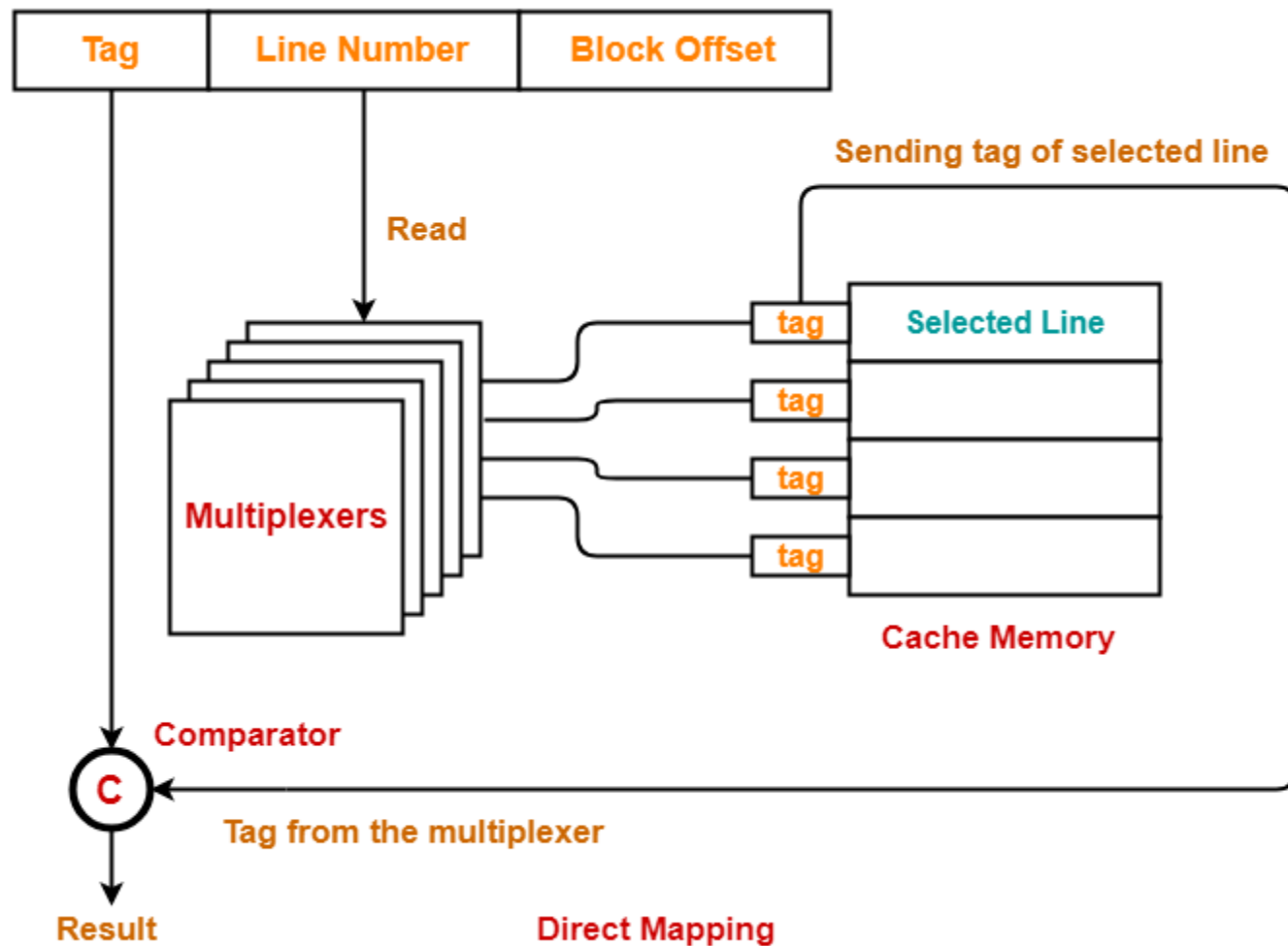


Tag	Line Number	Block / Line Offset
-----	-------------	---------------------



Block Number

Division of Physical Address in Direct Mapping



Example-Direct Mapping

- Cache memory size: 512 x 12
- Main memory size : 32K X 12
- Block size: 8 words

Identification of a block frame in a cache:

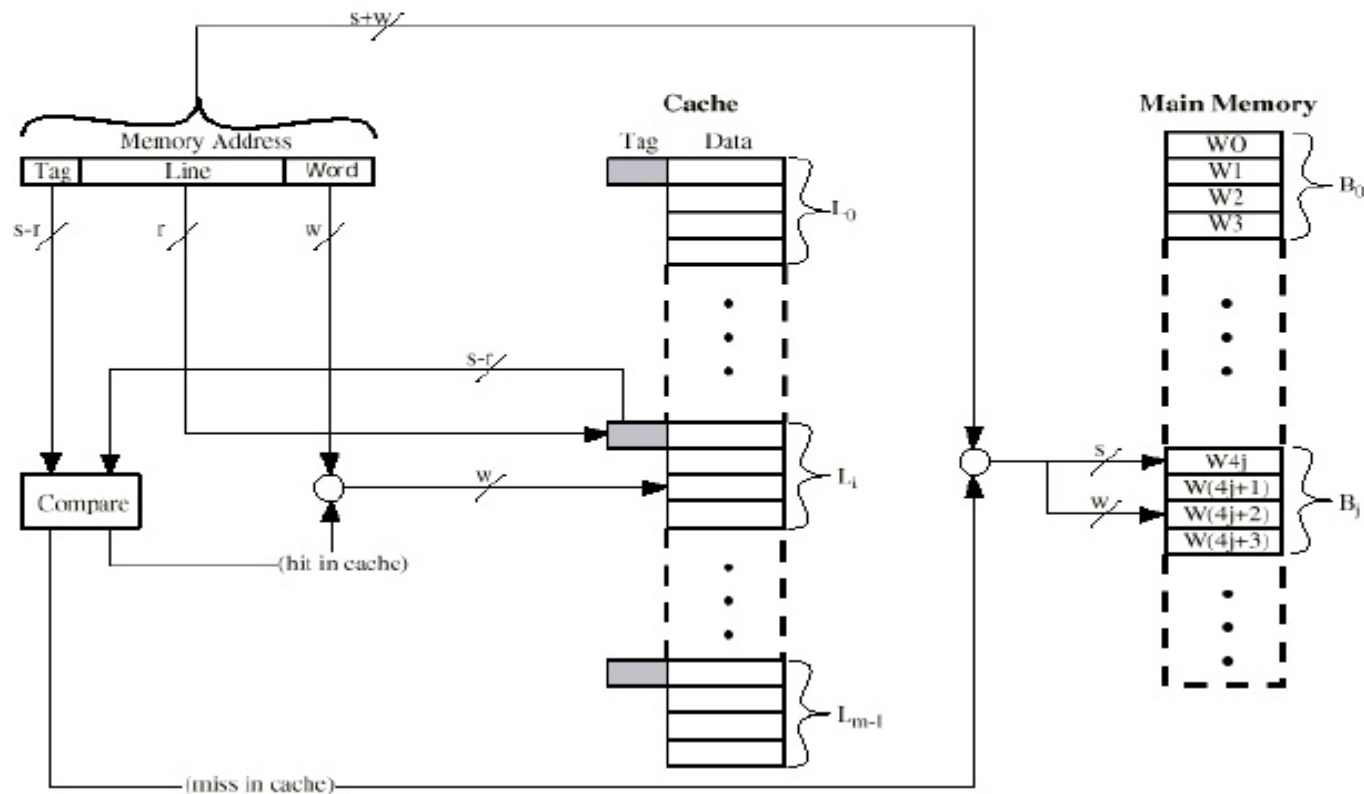
Address bit : 2^{15} [to address the main memory]

Number of blocks in cache memory : $512/8 = 64$
blocks

-
- Address bits to address cache memory = 9 [$2^9 = 512$]
 - CPU generated address bits: 15 bits
 - It is divided into 9 bits to address the cache [index] and 6 bits are saved as a tag in each block.
 - Index bits are further classified as cache block [line]=6 and word address [3]

Block Identification in Direct. mapping

Direct Mapping Cache Organization



Problem

- A digital computer has a memory unit of $64K \times 16$ and a cache memory of $1K$ word. The cache uses direct mapping with a block-size of four words.

- a) How many bits are there in the tag, index, block and word fields of the address format.
- b) How many blocks the cache can accommodate?

$16 \rightarrow 6, 10, 8, 2$

$1k/4 \rightarrow 256$

- Consider a direct mapped cache of size 16 KB with block size 256 bytes. The size of main memory is 128 KB. Find-
- Number of bits in tag
- Tag directory size
- $17 = \text{MM}$
- $14 = \text{CM} = \text{index}$
- $17 - 14 = 3$ bits for tag
- Number of tags * tag size
- Number of lines = 64
- $64 * 3 = 192$ bits

- Consider a direct mapped cache of size 512 KB with block size 1 KB. There are 7 bits in the tag. Find-
- Size of main memory
- Tag directory size
- $19 = \text{index}, \text{tag } 7 \text{ bits} = 26$
- $\text{MM} = 2^{26}$
- Number of lines = 512
- $512 * 7$
- 3584 bits

- 1. Consider a direct mapped cache with block size 4 KB. The size of main memory is 16 GB and there are 10 bits in the tag. Find-
 - Size of cache memory

- Tag directory size
- 2. Consider a direct mapped cache of size 32 KB with block size 32 bytes. The CPU generates 32 bit addresses. Find the number of bits needed for cache indexing and the number of tag bits.
- 3. Consider a machine with a byte addressable main memory of 2^{32} bytes divided into blocks of size 32 bytes. Assume that a direct mapped cache having 512 cache lines is used with this machine. The size of the tag field in bits is _____.

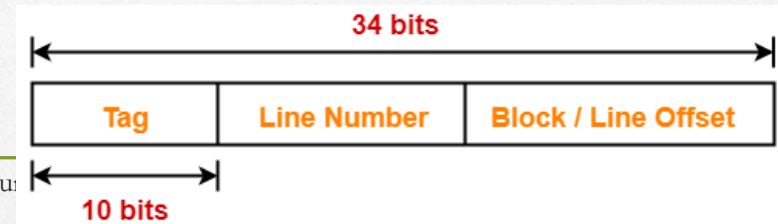
Size of main memory

= 16 GB

= 2^{34} bytes

Thus, Number of bits in physical address = 34 bits

- **Block size**
- = 4 KB
- = 2^{12} bytes
- Thus, Number of bits in block offset = 12 bits
- Number of bits in line number
- = Number of bits in physical address – (Number of bits in tag + Number of bits in block offset)
- = 34 bits – (10 bits + 12 bits)
- = 34 bits – 22 bits
- = 12 bits
- Thus, Number of bits in line number = 12 bits
- Size of cache memory
- = Total number of lines in cache x Line size
- = $2^{12} \times 4$ KB
- = 2^{14} KB
- = 16 MB
- Thus, Size of cache memory = 16 MB
- Tag directory size
- = Number of tags x Tag size
- = Number of lines in cache x Number of bits in tag
- = $2^{12} \times 10$ bits
- = 40960 bits
- = 5120 bytes
- Thus, size of tag directory = 5120 bytes

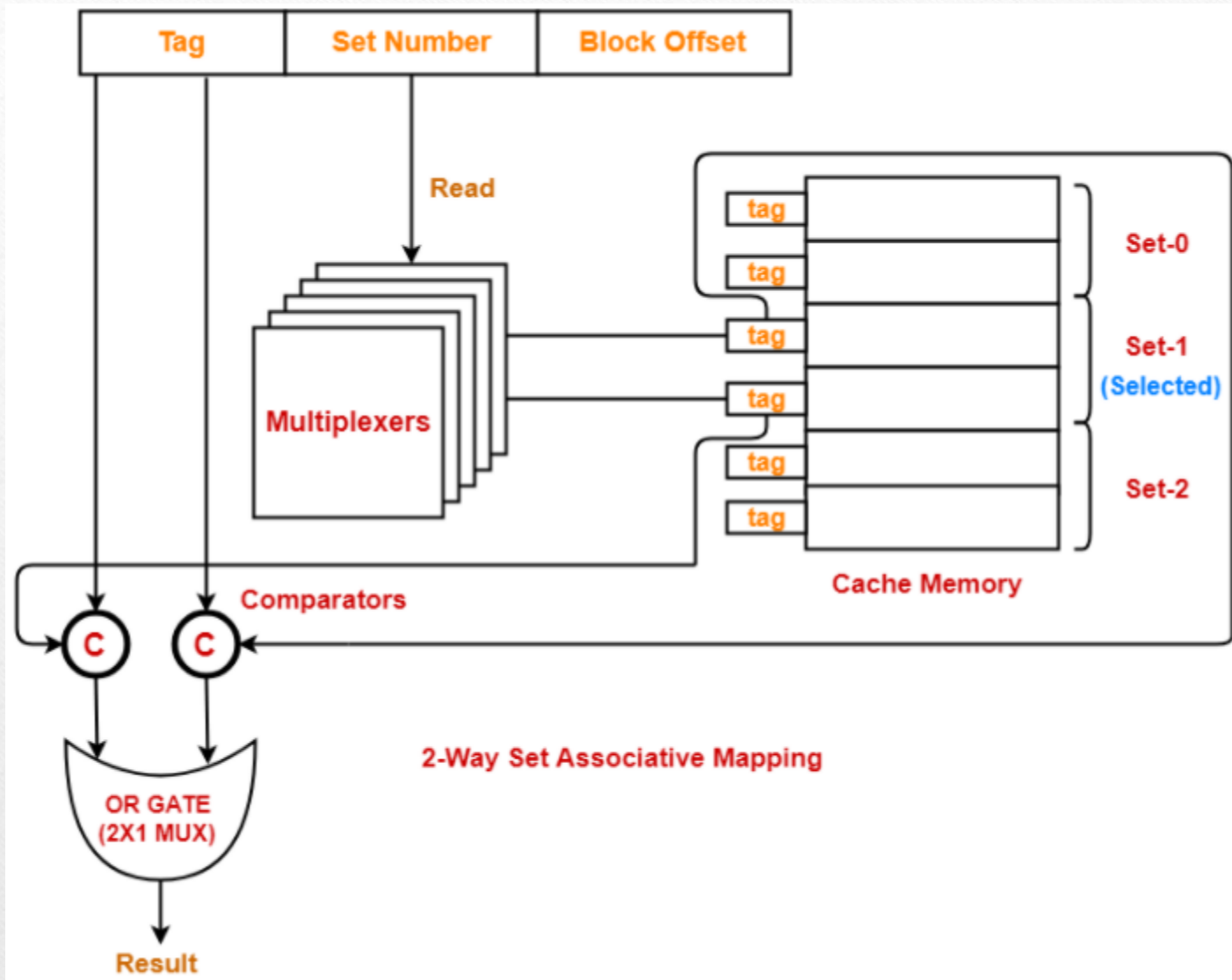


Set Associative Mapping

- Cache is divided into a number of sets
- Each set contains a number of lines
- A given block maps to any line in a given set
 - e.g. Block B can be in any line of set i
- e.g. 2 lines per set
 - 2 way associative mapping
 - A given block can be in one of 2 lines in only one set

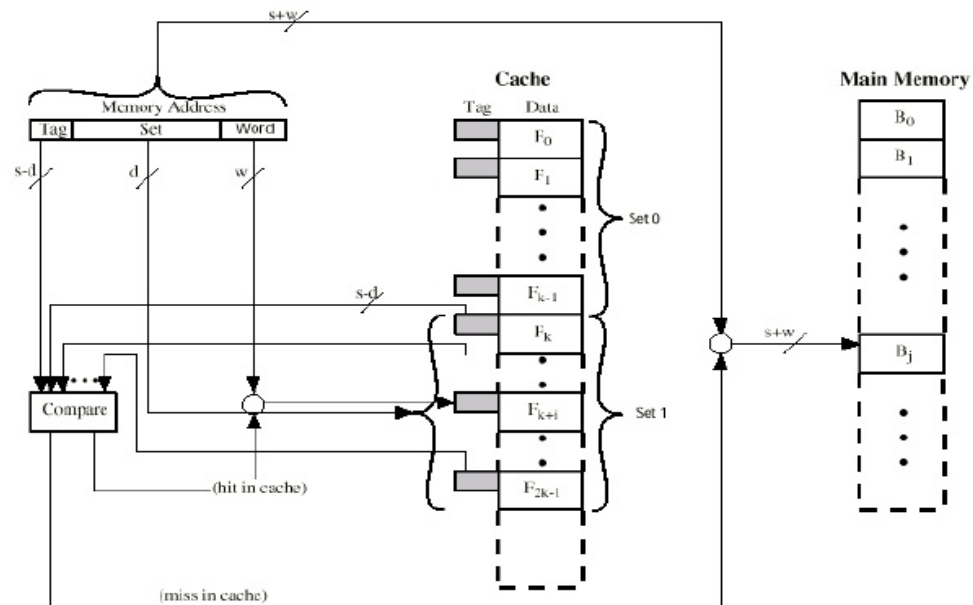


Division of Physical Address in K-way Set Associative Mapping



Block Identification in k-way set associative mapping

Two Way Set Associative Cache Organization



- Disadvantage of Direct mapping:

two words with same index in their address but with different tag values cannot reside in cache memory.

Advantage set associative:

hit ratio will improve as the set size increases.[more words with the same index but different tag values can reside in cache]

Disadvantage of set associative:

- complex comparison logic.

Problems

- A set associative cache consists of 64 lines or slots, divided into four line sets. Main memory consists 4k blocks of 128 words each. Show the format of main memory addresses.

The cache is divided into 16 sets of 4 lines each. Therefore, 4 bits are needed to identify the set number. Main memory consists of $4K = 2^{12}$ blocks. Therefore, the set plus tag lengths must be 12 bits and therefore the tag length is 8 bits. Each block contains 128 words. Therefore, 7 bits are needed to specify the word.

	TAG	SET	WORD
Main memory address =	8	4	7

Problem 2

- A two-way set associative cache has lines of 16 bytes and a total size of 8k bytes. The 64-Mbyte main memory is byte addressable. Show the format of main memory addresses.

There are a total of $8 \text{ kbytes} / 16 \text{ bytes} = 512$ lines in the cache. Thus the cache consists of 256 sets of 2 lines each. Therefore 8 bits are needed to identify the set number. For the 64-Mbyte main memory, a 26-bit address is needed. Main memory consists of $64\text{-Mbyte} / 16 \text{ bytes} = 2^{22}$ blocks. Therefore, the set plus tag lengths must be 22 bits, so the tag length is 14 bits and the word field length is 4 bits.

	TAG	SET	WORD
Main memory address =	14	8	4

1. Consider a 2-way set associative mapped cache of size 16 KB with block size 256 bytes. The size of main memory is 128 KB. Find-

- Number of bits in tag
 - Tag directory size
-

2. Consider a 8-way set associative mapped cache of size 512 KB with block size 1 KB. There are 7 bits in the tag. Find-

- Size of main memory
- Tag directory size

3. Consider a 4-way set associative mapped cache with block size 4 KB. The size of main memory is 16 GB and there are 10 bits in the tag. Find-

- Size of cache memory
- Tag directory size

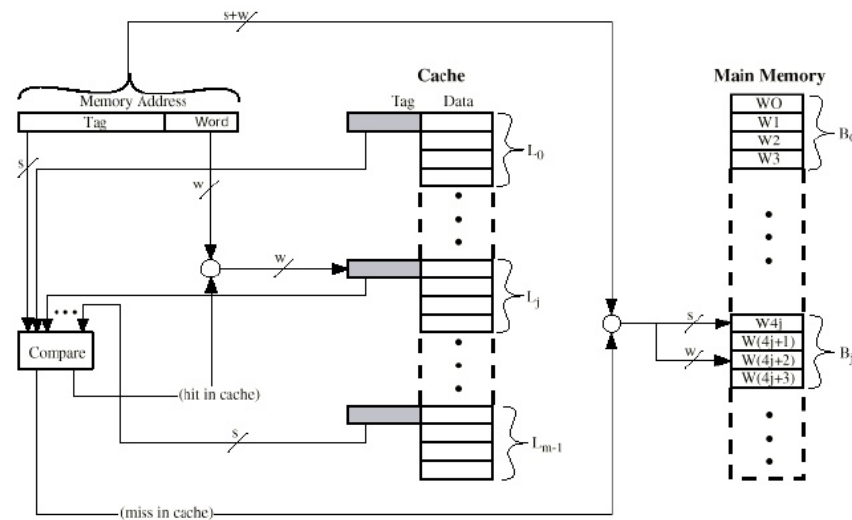
Fully Associative Mapping

- A main memory block can load into any line of cache
- Memory address is interpreted as tag and word
- Tag uniquely identifies block of memory
- Every line's tag is examined for a match
- Cache searching gets expensive

Tag / Block Number	Block Offset
---------------------------	---------------------

Block Identification in Fully associative mapping

Fully Associative Cache Organization



Problem 3

- Consider a cache consisting of 128 blocks of 16 words each, for a total of 2k words, and assume that the main memory is addressable by a 16 bit address and it consists of 4k blocks. Show the format of main memory address in all three types of mapping.
- Assume 2way set associative for set associative mapping.

Direct:

Tag $\rightarrow$ 5bits $\rightarrow 2^{12}/2^7 = 2^5$

Block $\rightarrow$ 7bits $\rightarrow 2^7 = 128$

Word $\rightarrow$ 4bits $\rightarrow 2^4 = 16$ words in a block



Associative:



Set associative:

Tag $\rightarrow$ 6bits $\rightarrow 2^{12}/2^6 = 2^6$

Set $\rightarrow$ 6bits $\rightarrow 2^6 = 64$ sets

Word $\rightarrow$ 4bits $\rightarrow 2^4 = 16$ words in a block



problem4

- A block set associative cache consists of a total of 64-lines divided into four-line sets. The main memory contains 4096 blocks, each consisting of 128 words.
- What is the main memory address size?
- Format of main memory address?
- What is the size of cache memory?

-
- What is the main memory address size?
 - Total word = $4096 * 128 = 2^{19}$
 - Address size = 19
 - Format of main memory address?
 - Tag $\rightarrow$ 8bits, set $\rightarrow$ 4bits, word $\rightarrow$ 7bits
 - What is the size of cache memory? $64 * 128$

- Consider a fully associative mapped cache of size 16 KB with block size 256 bytes. The size of main memory is 128 KB. Find-
 - Number of bits in tag
 - Tag directory size
- Consider a fully associative mapped cache of size 512 KB with block size 1 KB. There are 17 bits in the tag. Find-
 - Size of main memory
 - Tag directory size

Sources of Cache Misses (Three C's)

Compulsory Misses: These are misses that are caused by the cache being empty initially or by the first reference to a location in memory. Sometimes referred to as **Cold misses**.

Capacity Misses : If the cache cannot contain all the blocks needed during the execution of a program, capacity misses will occur due to blocks being discarded and later retrieved.

Conflict Misses: If the cache mapping is such that multiple blocks are mapped to the same cache entry. Common in set associative or direct mapped block placement, where a block can be discarded and later retrieved if too many blocks map to its set. Also called collision or interference misses.

Block Replacement

- **Least Recently Used: (LRU)**

Replace that block in the set that has been in the cache longest with no reference to it.

- **First Come First Out: (FIFO)**

Replace that block in the set that has been in the cache longest.

- **Least Frequently Used: (LFU)**

Replace that block in the set that has experienced the fewest references

Update Policies - Write Through

- Update main memory with every memory write operation
- Cache memory is updated in parallel if it contains the word at specified address.
- Advantage:
 - main memory always contains the same data as the cache
 - easy to implement
- Disadvantage: -
 - write is slower
 - every write needs a main memory access

Write Back

- Only cache is updated during write operation and marked by flag. When the word is removed from the cache (at the time of replacement), it is copied into main memory
-

- *Advantage:*

- writes occur at the speed of the cache memory
- multiple writes within a block require only one write to main memory

- *Disadvantage:*

- harder to implement
- main memory is not always consistent with cache

Update policies – Contd..

- Write-Allocate
 - update the item in main memory and bring the block containing the updated item into the cache.
- Write-Around or Write-no-allocate
 - correspond to items not currently in the cache (i.e. write misses) the item is updated in main memory only without affecting the cache.

Update policies – Contd..

- Write back:-Write only in cache, updating main memory only at the time of replacement.
- Write through:-Both are updating for each write operation.
- Write-Allocate:- first in main memory, then copy the block into cache.
- Write-Around or Write-no-allocate:- when write miss occurred, updated in main memory without affecting the cache.

Performance analysis

- Look through: The cache is checked first for a hit, and if a miss occurs then the access to main memory is started.
- Look aside: access to main memory in parallel with the cache lookup.

- **Look through**

$$T_A = T_C + (1-h)*T_M$$

T_A – Average read access time

T_C is the average cache access time

T_M is the average main memory access time

- **Look aside**

$$T_A = h*T_C + (1-h)*T_M$$

number of references found in the cache

- hit ratio $h = \frac{\text{number of references found in the cache}}{\text{total number of memory references}}$

- **Miss Ratio $m=(1-h)$**

Example: assume that a computer system employs a cache with an access time of 20ns and a main memory with a cycle time of 200ns. Suppose that the hit ratio for reads is 90%,

- a) what would be the average access time for reads if the cache is a look through-cache?

$$\begin{aligned}\text{The average read access time } (T_A) &= T_c + (1-h)*T_M \\ 20\text{ns} + 0.10*200\text{ns} &= 40\text{ns}\end{aligned}$$

- b) what would be the average access time for reads if the cache is a “look-Aside” cache?

$$\begin{aligned}\text{The average read access time in this case } (T_A) \\ = h*T_c + (1-h)*T_M &= 0.9*20\text{ns} + 0.10*200\text{ns} = 38\text{ns}\end{aligned}$$

Problem 1

- Consider a memory system with $T_c = 100\text{ns}$ and $T_m = 1200\text{ns}$. If the effective access time is 10% greater than the cache access time, what is the hit ratio H in look-through cache?

$$\Rightarrow T_A = T_C + (1-h)*T_M$$

$$\Rightarrow 1.1 T_c = T_c + (1-h)*T_M$$

$$\Rightarrow 0.1T_C = (1-h)* T_M$$

$$\Rightarrow 0.1 * 100 = (1-h) * 1200$$

$$\Rightarrow 1-h = 10/1200$$

$$\Rightarrow h = 1190/1200$$

Problem 2

- A computer system employs a write-back cache with a 70% hit ratio for writes. The cache operates in look-aside mode and has a 90% read hit ratio. Reads account for 80% of all memory references and writes account for 20%. If the main memory cycle time is 200ns and the cache access time is 20ns, what would be the average access time for all references (reads as well as writes)?

Total Reference

100%			
80% Reads		20 %Writes	
90% hit	10% Miss	70 hit	30 Miss

The average access time for reads

$$= 0.9 \times 20\text{ns} + 0.1 \times 200\text{ns} = 38\text{ns}.$$

The average write time

$$= 0.7 \times 20\text{ns} + 0.3 \times 200\text{ns} = 74\text{ns}$$

Hence the overall average access time for combined reads and writes is

$$= 0.8 \times 38\text{ns} + 0.2 \times 74\text{ns} = 45.2\text{ns}$$

References

- J. L. Hennessy & D.A. Patterson, Computer architecture: A quantitative approach, Fourth Edition, Morgan Kaufman, 2004.